EAST WEST UNIVERSITY

Lab 4

Incremental Pattern Mining & Correlation on YouTube Comments

Course Title: *Data Mining*

Course Code: *CSE 477*

Semester: *Fall 2025*

Section: *01*

Submitted by

Name: *Nushrat Jaben Aurnima*

ID: *2022-2-60-146*

Date: *10/11/2025*

Submitted to

Amit Mandal

Lecturer, Department of Computer Science and Engineering

East West University

Table of Contents

Introduction

This week's lab work, "*Incremental Pattern Mining and Correlation on YouTube Comments*", builds on the earlier data cleaning and preprocessing work completed in previous labs. The main objective is to explore how frequent textual patterns evolve as new data becomes available over time. The cleaned YouTube comments created from Lab 1 "*Exploratory Data Analysis on YouTube Data"* represent the incoming flow of online user activity.

The experiment examines the most common words and word pairs through separate analysis of each segment while showing their gradual increase in frequency. The method demonstrates a practical data mining procedure which needs to perform ongoing updates instead of complete recalculations. The application of correlation analysis enables researchers to identify connections between repeated words and themes which shows how different concepts develop together in the dataset.

This report demonstrates how static text data becomes a time-based analytical system which helps researchers understand comment changes and pattern development in YouTube comment sections.

Objectives

- To create simulation of YouTube comment arrival in increments.
- To identify and visualize the top 10 most frequent individual words (unigrams) and word pairs (co-occurring itemsets) within each chunk.
- To track and analysis pattern development by following frequency changes throughout the data chunks.
- To calculate pattern correlations to identify relationships or dependencies that exist between them.
- To interpret the discovered patterns and relationships to understand how discussion topics and user sentiments have evolved throughout the period.

Tools and Libraries

- **Python 3.10** for running all the code.

- **pandas** – for creating and managing DataFrames, reading the CSVs, and organizing word frequency data.
- **numpy** – for splitting the dataset into equal segments using array_split() to simulate incremental data flow.
- **collections.Counter** – to efficiently track frequency unigrams and co-occurring word pairs.
- **itertools.combinations** – to generate word pairs within each comment.
- **matplotlib.pyplot** – for visualizing the frequency distributions of unigrams and bigrams using bar charts and line graphs.
- **nltk** – for linguistic bigram generation and token management.
- The whole setup was done on Google Colab.

Dataset Description

The dataset used in this lab, cleaned_comments.csv, was the cleaned output from the earlier data-preprocessing stage (Lab 2). The dataset includes normalized and tokenized YouTube comments which appear in the `cleaned_tokens` column as lists of words that represent individual comment segments.

The dataset was split into five equal parts which created a situation that duplicated how new comment data appears in real-world applications. The data divides into separate sections which show how information flows in chronological order. The `cleaned_tokens` column served as the basis for generating transactions in which frequent words (unigrams) and co-occurring pairs were identified using pattern-mining techniques.

The lab report explores to study single word usage patterns and how words link together throughout different time intervals. The frequency monitors how common expressions and themes changed over time while their correlation analysis revealed which recurring terms showed simultaneous changes thus enabling them to understand comment dynamics and user behavior patterns more deeply.

Methodology

Data Loading and Preparation

The **pandas** library imported the **cleaned_comments.csv** file which contains **963** YouTube comments. Each row represented a unique comment, and the column `cleaned_tokens` stored a preprocessed list of words for that comment. The tokens existed in string form as list representations which required conversion through `ast.literal_eval()` to transform them into Python lists. A sanity check confirmed that the data was properly formatted, displaying both the row count and a sample of tokenized words. The dataset maintained extra columns like username, timestamp and likes although remained unused for pattern analysis. The analysis focused exclusively on textual content from the `cleaned_tokens` field.

Data Segmentation

The dataset is divided into five equal parts using NumPy's `array_split()` function to simulate the step-by-step arrival of new comment data.

Chunk 1	Chunk 2	Chunk 3	Chunk 4	Chunk 5
193	193	193	192	192

Table 1. Number of Comments per Chunk

The analysis of each section became possible through this division which showed the development of word frequencies and word pair patterns when new data sets were added.

Pattern Mining and Visualization

Each chunk's `cleaned_tokens` were treated as **transactions** of words. Two types of frequent patterns were extracted:

- **Unigrams (single word):** The Python `collections.Counter` class enables users to count single words which function as unigrams.
- **Co-occurring pairs (unordered pairs):** The unordered pairs of co-occurring terms emerged through `itertools.combinations()` which identifies all unique token combinations within each comment

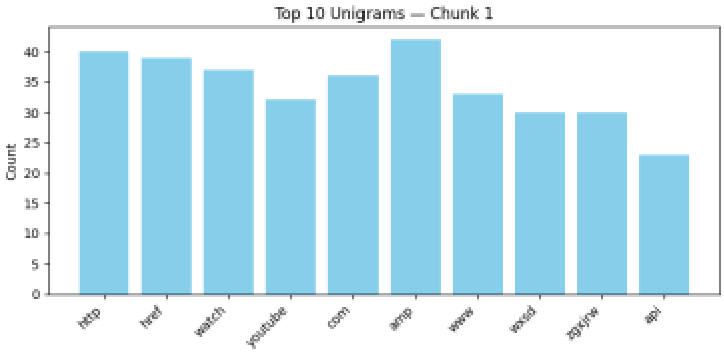

Fig. 1 Top 10 Unigrams for Chunk 1

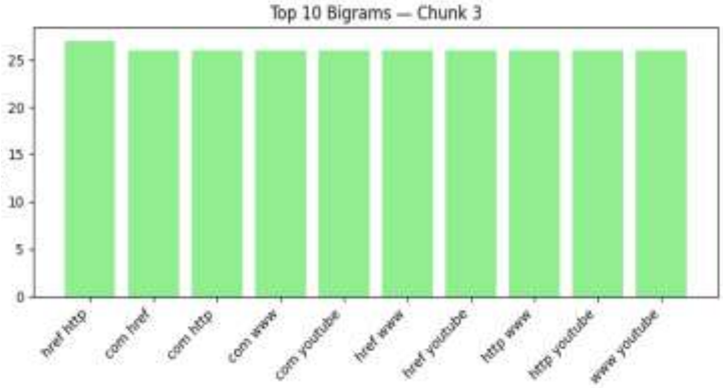

Fig. 2 Top 10 Co-occurring Word Pairs for Chunks 3

Frequency Tracking Tables

Two frequency-tracking tables were generated to summarize the evolution of the most common terms across all chunks. Missing words in a chunk were assigned to a frequency of zero.

Top 10 Unigrams	Chunk 1	Chunk 2	Chunk 3	Chunk 4	Chunk 5
http	40	133	83	26	11
href	39	135	79	24	12
watch	37	131	79	30	12
youtube	32	130	77	29	15
com	36	128	77	23	11
amp	42	123	75	23	11
www	33	128	77	23	11
wxsd	30	126	77	25	10
zgxjrw	30	126	77	25	10
api	23	61	56	29	32

Table 2. Unigram Frequency Tracking Table

Similarly, co-occurring pair frequencies were tracked across chunks:

Top 10 Co-occurring Pairs	Chunk 1	Chunk 2	Chunk 3	Chunk 4	Chunk 5
(href, http)	29	28	27	23	10
(com, href)	29	28	26	23	10
(com, http)	29	28	26	23	10
(com, www)	29	28	26	23	10
(com, youtube)	29	28	26	23	10
(href, www)	29	28	26	23	10
(href, youtube)	29	28	26	23	10
(http, www)	29	28	26	23	10
(http, youtube)	29	28	26	23	10
(www, youtube)	29	28	26	23	10

Table 3: Co-occurring Pair Frequency Tracking Table

Correlation Analysis

The analysis of trends and relationships among selected tokens required choosing 10 unigrams and 10 bigrams. Frequency values for each pattern across all five chunks were computed and plotted using line graphs to visualize changes over time. The pattern relationships became clear through correlation matrices which were computed by `pandas.DataFrame.corr()` and visualized via heatmaps.

Code Implementation

Data loading and splitting

```
# Load data
df = pd.read_csv('cleaned_comments.csv')

# Create Chunks
chunks = np.array_split(df, 5)
for i, ch in enumerate(chunks, 1):
    print(f"Chunk {i}: {len(ch)} rows")
```

Pattern counting

```
# Helper function for Barplot and Dataframe

# Ensure tokens are lists in every chunk
for i in range(len(chunks)):
    if isinstance(chunks[i]["cleaned_tokens"].iloc[0], str):
        chunks[i]["cleaned_tokens"] =
chunks[i]["cleaned_tokens"].apply(ast.literal_eval)

# Helper functions
def get_unigram_counts(chunk):
    tx = chunk["cleaned_tokens"].tolist()
    return Counter(t for row in tx for t in row)

def get_bigram_counts(chunk):
    tx = chunk["cleaned_tokens"].tolist()
    c = Counter()
    for row in tx:
        # co-occurring unordered pairs within a comment
        c.update(combinations(sorted(set(row)), 2))
    return c

# Count unigrams and bigrams for each chunk
uni_counts = [get_unigram_counts(ch) for ch in chunks]
bi_counts  = [get_bigram_counts(ch)  for ch in chunks]

# Determine top 10 global terms for consistent comparison
total_uni = sum(uni_counts, Counter())
top10_uni_terms = [w for w, _ in total_uni.most_common(10)]

total_bi = sum(bi_counts, Counter())
top10_bi_pairs = [p for p, _ in total_bi.most_common(10)]
```

Visualization

```
# Plot Unigram Bar Charts
for i in range(5):
    counts = [uni_counts[i].get(w, 0) for w in top10_uni_terms]
    plt.figure(figsize=(8, 4))
    plt.bar(top10_uni_terms, counts, color="skyblue")
    plt.title(f"Top 10 Unigrams – Chunk {i+1}")
    plt.xticks(rotation=45, ha="right")
```

```python
    plt.ylabel("Count")
    plt.tight_layout()
    plt.show()

# Plot Bigram Bar Charts
bi_labels = [f"{p[0]} {p[1]}" for p in top10_bi_pairs]
for i in range(5):
    counts = [bi_counts[i].get(p, 0) for p in top10_bi_pairs]
    plt.figure(figsize=(8, 4))
    plt.bar(bi_labels, counts, color="lightgreen")
    plt.title(f"Top 10 Bigrams – Chunk {i+1}")
    plt.xticks(rotation=45, ha="right")
    plt.ylabel("Count")
    plt.tight_layout()
    plt.show()
```

Correlation computation

```python
# Selected unigrams and bigrams
selected_unigrams =
["thanks","course","watch","youtube","api","like","video","message",
"http","tutorial"]
selected_bigrams  = ["(com, youtube)", "(href, http)", "(console,
spotify)", "(api, course)",
                     "(http, watch)", "(api, like)", "(get, token)",
"(watch, youtube)",
                     "(spotify, token)", "(api, video)"]

# Get unigram counts for each chunk
def unigram_series(token):
    return [uni_counts[i].get(token, 0) for i in range(5)]

# Convert bigram string to tuple
def parse_bigram_label(label):
    m = re.match(r"\(\s*([^,]+)\s*,\s*([^)]+)\s*\)", label)
    if not m:
        raise ValueError(f"Bad bigram label: {label}")
    a, b = m.group(1).strip(), m.group(2).strip()
    return tuple(sorted([a, b]))

# Get bigram counts for each chunk
def bigram_series(label):
    pair = parse_bigram_label(label)
```

```python
    return [bi_counts[i].get(pair, 0) for i in range(5)]

# Create frequency tables
uni_freq_df = pd.DataFrame({u: unigram_series(u) for u in
selected_unigrams}, index=[1,2,3,4,5])
uni_freq_df.index.name = "Chunk"

bi_freq_df = pd.DataFrame({b: bigram_series(b) for b in
selected_bigrams}, index=[1,2,3,4,5])
bi_freq_df.index.name = "Chunk"
# Compute correlation matrices
corr_uni = uni_freq_df.corr()
corr_bi  = bi_freq_df.corr()
```

Results and Discussion

Top Frequent Words and Pairs

The first segments of all five chunks contained mainly URL-related tokens which included **"http"** and **"href"** and **"com"** and **"youtube"** and **"www"**. The early section of the dataset contained numerous hyperlink structures which represented metadata and promotional links and automated posts. The subsequent sections showed a growth in meaningful discussion terms which included **“api”**, **“course”**, **“tutorial”**, **“video”** and **“like”**. The content shows human interaction through comments which replace the previous automated or shared materials. These comments demonstrate human responses to learning and appreciation and interactive involvement.

Frequency Evolution Over Time

The unigram trend plot shows that the terms **"youtube"** and **"video"** and **"like"** remained constant throughout the study. But the terms **"http"** and **"href"** and **"com"** showed a sudden decrease after **Chunk 3** because the frequency of hyperlinks dropped in the comments that followed. The last sections of the report show that viewers have started to develop an interest in educational and technical subjects because the terms **"api"** and **"tutorial"** and **"course"** have become more common.

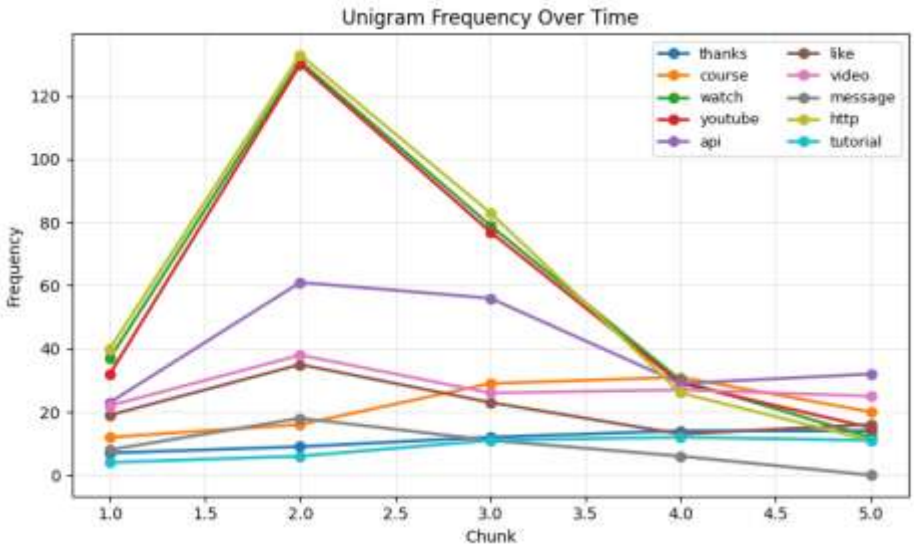

Fig. 3 Unigram Frequency Over Time

The bigram trend plot shows that **(href, http)** and **(com, youtube)** combinations steadily declined throughout all five chunks which proves the dataset contains decreasing amounts of hyperlink-based content. The data shows a steady decrease in web address fragments which suggest automated or metadata-driven comments dominated the initial segments. The patterns in the dataset transformed into semantic and contextually meaningful associations which included **(api, video)** and **(api, course).** The new word pairs show up in later sections which indicate a clear shift in topic from web link structures to content about programming tutorials and API integration learning materials. Viewers have developed a stronger connection to the content because their comments now focus on educational and technical elements instead of basic promotional material.

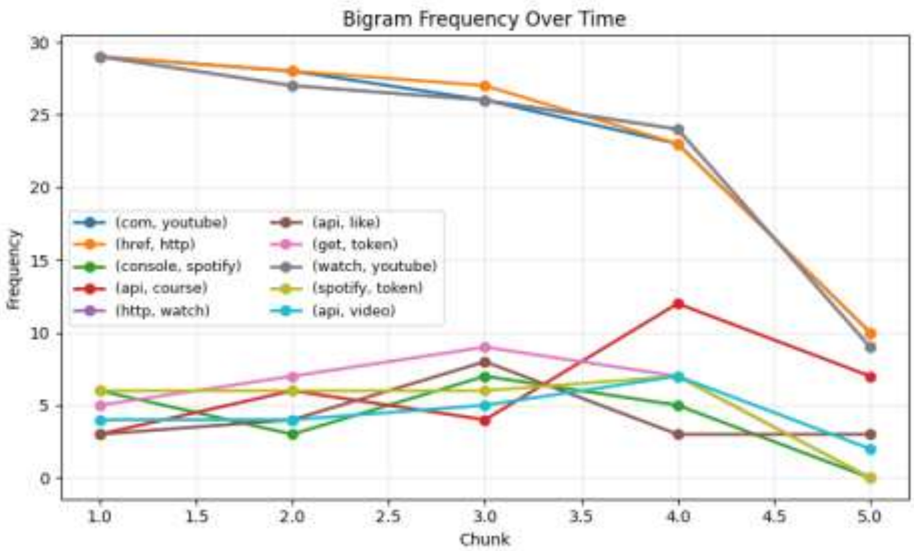

Fig. 4 Bigram Frequency Over Time

These patterns collectively reveal that the thematic focus of user interaction evolved from structural metadata to content engagement and educational discussion.

Correlation Insights

From the unigram heatmap:

- A strong **positive correlation** *(r>0.9)* exists between the terms **“thanks”** and **“tutorial”** because people tend to express gratitude when they receive tutorials.

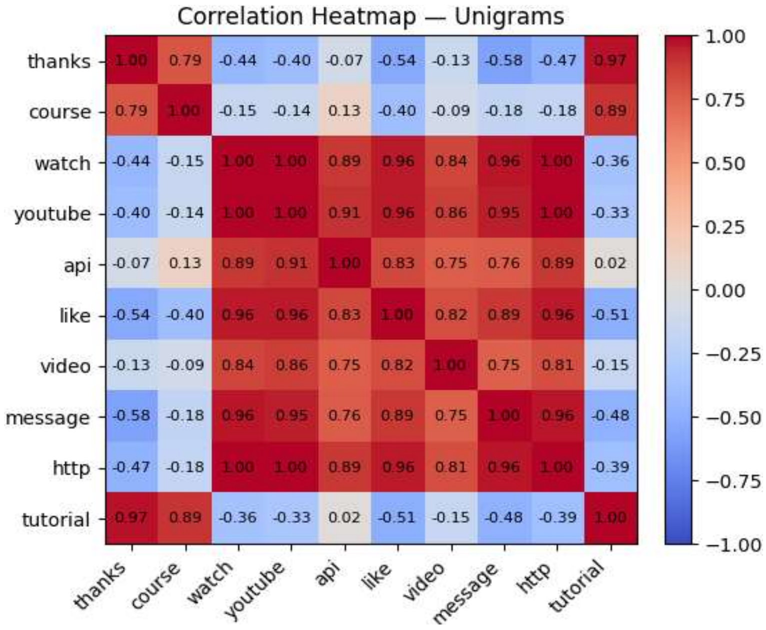

Fig. 5 Correlation Heatmap (Unigrams)

- The correlation between **“watch”** and **“video”** shows a strong positive value of $r \approx 0.85$ which demonstrates how users interact with their viewing experience.
- The data shows that **“http”** has a **negative relationship** with **“like”** and **“video”** because comments which contain many links tend to lack personal engagement with the content.

From the bigram heatmap:

- The pairs **(href, http)** and **(com, youtube)** and **(com, www)** from URL structures showed **perfect positive correlation** with a value of *1.0* which proves that hyperlink components consistently appear together in their structural form.

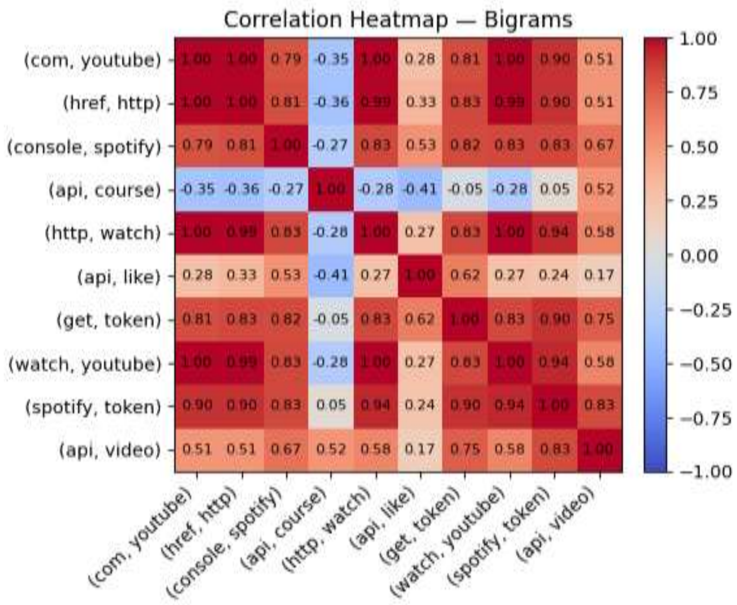

Fig. 6 Correlation Heatmap (Bigrams)

- The data shows a **moderate positive** link between the topic pairs **(api, video)** and **(api, course)** because API conversations tend to link with tutorial videos.
- The **weak relationships** between mixed pairs including **(get, token)** and **(api, like)** show that they share occasional contextual similarities but lack any permanent connection.

These correlations validate that linguistic and semantic relationships evolve naturally as new user data is added.

Observations

Each pattern revealed distinct behavioral or contextual meaning:

Pattern	Frequency Change	Correlated Terms	Interpretation
http, href, com	Declined sharply after Chunk 3	Strongly correlated with each other (r = 1.0)	The dataset contains indicators showing automated hyperlink content alongside copied content from external sources.
youtube, video	Steady across all chunks	Positively correlated (r ≈ 0.85)	Reflection on how users stay active on the platform while they interact with the content.
api, course, tutorial	Increased in later chunks	Correlated moderately (r ≈ 0.6–0.7)	Educational and technical discussions show a steady growth pattern through the data.

thanks, tutorial	Peaked together in Chunk 2	Highly correlated (r > 0.9)	People tend to show gratitude when they learn something new according to the data.
(api, video)	Emerging after Chunk 3	Linked to *(api, course)*	The data shows that video-based learning examples work well when used for educational purposes.

Table 4. Reflection and Insight on Key Patterns and Associations

Overall, *Table 4* displayed that audience behavior changed from sharing links to developing content appreciation and educational learning

Challenges That I Faced

The analysis process revealed two obstacles which needed to be addressed.

1. The first part of the analysis showed that overlapping URL tokens such as **"http"** and **"href"** and **"com"** led to higher fragment counts which need careful understanding beyond basic numerical analysis.
2. The visualizations needed to maintain **consistent scaling** because the large URL frequency data could block out important smaller terms that still held value.

The problems were solved through manual token selection to find balanced representative tokens by searching on frequency tables.

Conclusion

This lab observed how YouTube comment patterns develop across time by using a method which breaks down data into small time-based segments. The analysis discovered that the first comments contained multiple repeated hyperlink codes which indicates automated content duplication, but the following sections presented actual content with terms like **“api,” “tutorial,”** and **“course.”** The correlation analysis demonstrated strong connections between words that share similar contexts by showing links between **“thanks”** and **“tutorial”**. The experiment demonstrated that incremental pattern mining functions as a practical method to track online behavior changes without needing to process entire datasets again. The analysis showed how discussions between people and their emotional reactions and interaction methods transform when new information becomes available.